



Visió Artificial

1st Week: Tutorial on skimage and Image processing

Kaisar Kushibar
kaisar.kushibar@ub.edu
Universitat de Barcelona

Julio Cezar Silveira Jacques-Junior
julio.silveira@ub.edu
Universitat de Barcelona

University of Barcelona

Mathematics and Computer Science department



Agenda

- Introduction to lab sessions
 - Calendar
 - Requirements
 - Evaluation
- Tools and materials
 - Environments
 - Python
 - Computer Vision Libraries

Visió Artificial

- Teaching team:

- Coordinator: Petia Ivanova Radeva (petia.ivanova@ub.edu)
- Kaisar Kushibar (kaisar.kushibar@ub.edu)
- Julio Cezar Silveira Jacques-Junior (julio.silveira@ub.edu)

- Course schedule:

- Group A: Wednesday **17:00 – 19:00** ID – Practice – Kaisar
- Group F: Wednesday **17:00 – 19:00** IF – Practice – Julio
- Group B: Thursday **17:00 – 19:00** ID – Practice – Kaisar

Calendar

Teoria (Dx 15-17) Presentacions pptx		Pràctiques (17-19h)						
Data	Contingut	Dimecres	Dijous	Grup A - Kaisar ID	Grup B - Kaisar ID	Grup F - Julio IF	Dx - A F	Dj - B
16. 09	1 Presentació de l'assignatura	16. 09	17. 09	1a Tutorial on skimage & Image processing			Tutorial and P1	
23. 09	2 Operadors lineals. histograma	23. 09	24. 09	1b Introducción al procesamiento de imágenes			P1	
30. 09	3 Edges	30. 09	01. 10	1b Introducción al procesamiento de imágenes			P1	
07. 10	4 Image & video segmentation	07. 10	08. 10	2a Color Image processing			Eval P1 & P2	
14. 10	5 HOG	14. 10	15. 10	2b Edges			P2	
21. 10	6. Image descriptors SIFT	21. 10	22. 10	3a Video segmentation and Template matching			Eval P2 & P3	
28. 10	7 Texture analysis	28. 10	29. 10	3a Video segmentation and Template matching			P3	
03. 11	Examen parcial (30.10-5.11)	03. 11						
11. 11	8 Face detection	11. 11	12. 11	4 HOG and SIFT, Texture analysis			Eval P3 & P4	
18. 11	9 Face recognition	18. 11	19. 11	4 HOG and SIFT, Texture analysis			P4	
25. 11	10 CNNs	25. 11	26. 11	5 Face detection			Eval P4 & P5	
02. 12		02. 12	03. 12	5 Face recognition			P5	
09. 12	13 Miniprojects	09. 12	10. 12	5 Face recognition			P5	
16. 12	14 Miniprojects	16. 12	17. 12				Eval P5	

Requirements

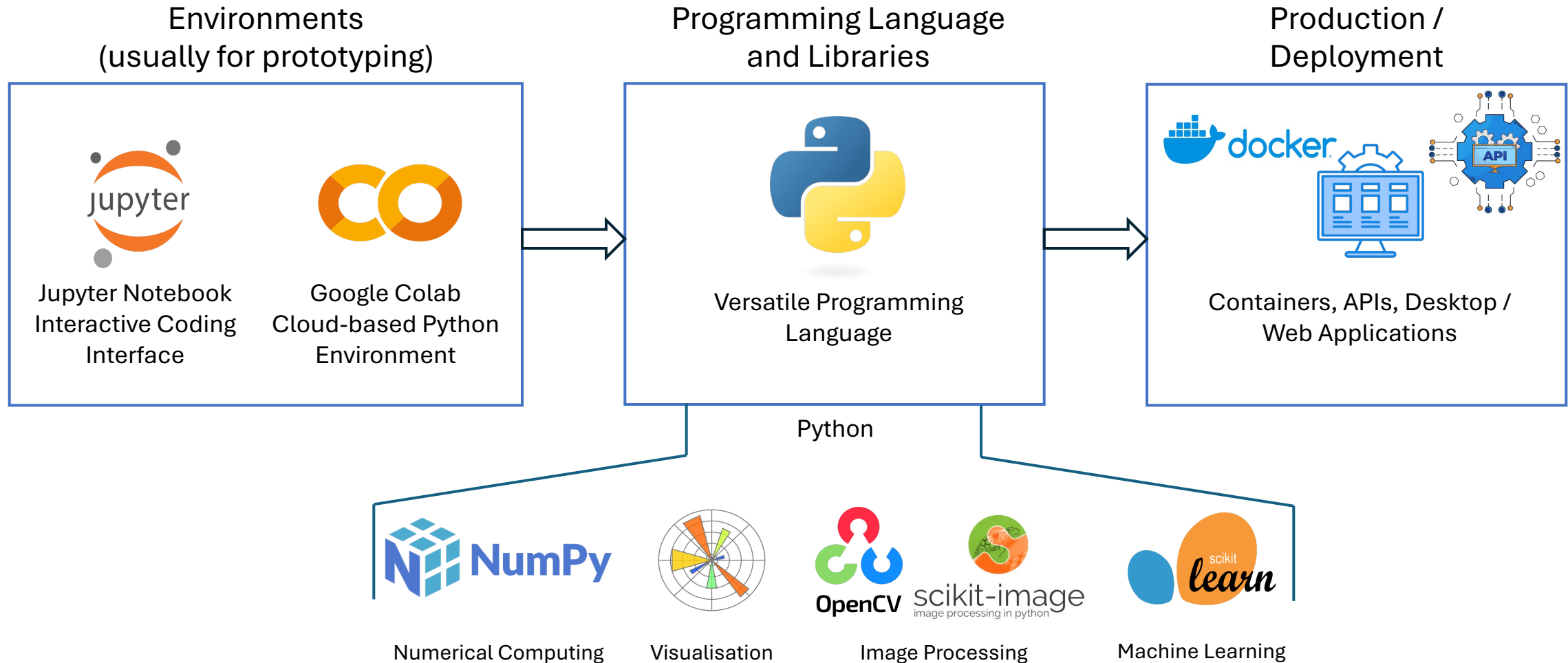
- The requirements for the course are based on:
 - linear algebra,
 - vector calculus,
 - statistics, and
 - numerical analysis
- Course does not assume prior imaging experience
- Programming (obviously)

Evaluation

- Jupyter Notebooks (6 laboratories)
 - These can be done in groups
 - **Not evaluated**
 - Use them to understand the topic
- Individual practical exams for each laboratory (45 minutes)
- 5 evaluations (practical exam)
 - No internet
 - Lab notebooks are allowed

Pràctiques (17-19h)						
Dimecres	Dijous	Grup A - Kaisar ID	Grup B - Kaisar ID	Grup F - Julio IF	Dx - A F	Dj - B
16. 09	17. 09	1a Tutorial on skimage & Image processing			Tutorial and P1	
23. 09	24. 09	1b Introducció al procesament de imàgenes			P1	
30. 09	01. 10	1b Introducció al procesament de imàgenes			P1	
07. 10	08. 10	2a Color Image processing			Eval P1 & P2	
14. 10	15. 10	2b Edges			P2	
21. 10	22. 10	3a Video segmentation and Template matching			Eval P2 & P3	
28. 10	29. 10	3a Video segmentation and Template matching			P3	
03. 11						
11. 11	12. 11	4 HOG and SIFT, Texture analysis			Eval P3 & P4	
18. 11	19. 11	4 HOG and SIFT, Texture analysis			P4	
25. 11	26. 11	5 Face detection			Eval P4 & P5	
02. 12	03. 12	5 Face recognition			P5	
09. 12	10. 12	5 Face recognition			P5	
16. 12	17. 12				Eval P5	

Computer Vision Workflow



Python and Libraries



A fundamental library for **numerical computing** in Python. It provides support for **large, multi-dimensional arrays and matrices**, along with a collection of mathematical **functions to operate** on these arrays.



A library of programming functions mainly aimed at real-time **computer vision**. It is extensively used for **image and video analysis**, object detection, and more.



An open-source Python package for **image processing**, providing a wide range of algorithms for tasks like **filtering, segmentation, and feature detection**.



A comprehensive library for creating static, animated, and interactive **visualizations** in Python. It is widely used for generating various types of **plots, charts, and figures**.



A free software **machine learning** library for Python. It features various **classification, regression, and clustering algorithms**, and is designed to interoperate with Numpy

How to ace the exams?

- Learn to work with the documentation

 scikit-image

[User guide](#) [Examples](#) [API reference](#) [Release notes](#) [Development](#) [About](#)

0.25 (stable)

Section Navigation

skimage

skimage.color

skimage.data

skimage.draw

skimage.exposure

skimage.feature

skimage.filters

skimage.filters.rank

skimage.future

skimage.graph

skimage.io

skimage.measure

skimage.metrics

skimage.morphology

skimage.registration

skimage.restoration

skimage.segmentation

skimage.transform

skimage.util

License

API reference > skimage.io

skimage.io

Reading and saving of images and videos.

<code>concatenate_images</code>	Concatenate all images in the image collection into an array.
<code>imread</code>	Load an image from file.
<code>imread_collection</code>	Load a collection of images.
<code>imread_collection_wrapper</code>	
<code>imsave</code>	Save an image to file.
<code>load_sift</code>	Read SIFT or SURF features from externally generated file.
<code>load_surf</code>	Read SIFT or SURF features from externally generated file.
<code>pop</code>	Pop an image from the shared image stack.
<code>push</code>	Push an image onto the shared image stack.
<code>ImageCollection</code>	Load and manage a collection of image files.
<code>MultiImage</code>	A class containing all frames from multi-frame TIFF images.

On this page

`concatenate_images()`

`imread()`

`imread_collection()`

`imread_collection_wrapper()`

`imsave()`

`load_sift()`

`load_surf()`

`pop()`

`push()`

`ImageCollection`

`MultiImage`

This Page

[Show Source](#)

```
1 help(plt.plot)
[5] ✓ 0.0s Python

... Help on function plot in module matplotlib.pyplot:

plot(*args: 'float | ArrayLike | str', scalex: 'bool' = True, scaley: 'bool' = True, data: 'DataParamType' = None, **kwargs) -> 'list[Line2D]'
Plot y versus x as lines and/or markers.

Call signatures:

    plot([x], y, [fmt], *, data=None, **kwargs)
    plot([x], y, [fmt], [x2], y2, [fmt2], ..., **kwargs)

The coordinates of the points or line nodes are given by *x*, *y*.

The optional parameter *fmt* is a convenient way for defining basic
formatting like color, marker and linestyle. It's a shortcut string
notation described in the «Notes» section below.

>>> plot(x, y)          # plot x and y using default line style and color
>>> plot(x, y, 'bo')     # plot x and y using blue circle markers
>>> plot(y)              # plot y using x as index array 0..N-1
>>> plot(y, 'r+')        # ditto, but with red plusses

You can use '.Line2D' properties as keyword arguments for more
control on the appearance. Line properties and *fmt* can be mixed.
The following two calls yield identical results:

... If the color is the only part of the format string, you can
    additionally use any 'matplotlib.colors' spec, e.g. full names
    ('green') or hex strings ('#000000').

Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

(p312) → **p1_tutorial** python -m pydoc -p 8080
Server ready at http://localhost:8080/
Server commands: [b]rowser, [q]uit
server> █

```
This changes the default colormap as well as the colormap of the current
image if there is one. See 'help(colormaps)' for more information.

plot(*args: 'float | ArrayLike | str', scalex: 'bool' = True, scaley: 'bool' = True, data: 'DataParamType' = None, **kwargs) -> 'list[Line2D]'
Plot y versus x as lines and/or markers.

Call signatures:

    plot([x], y, [fmt], *, data=None, **kwargs)
    plot([x], y, [fmt], [x2], y2, [fmt2], ..., **kwargs)

The coordinates of the points or line nodes are given by *x*, *y*.

The optional parameter *fmt* is a convenient way for defining basic
formatting like color, marker and linestyle. It's a shortcut string
notation described in the «Notes» section below.

>>> plot(x, y)          # plot x and y using default line style and color
>>> plot(x, y, 'bo')     # plot x and y using blue circle markers
>>> plot(y)              # plot y using x as index array 0..N-1
>>> plot(y, 'r+')        # ditto, but with red plusses

You can use '.Line2D' properties as keyword arguments for more
control on the appearance. Line properties and *fmt* can be mixed.
The following two calls yield identical results:

>>> plot(x, y, 'go--', linewidth=2, markersize=12)
>>> plot(x, y, color='green', marker='o', linestyle='dashed',
...      linewidth=2, markersize=12)

When conflicting with *fmt*, keyword arguments take precedence.

**Plotting labelled data**

There's a convenient way for plotting objects with labelled data (i.e.
data that can be accessed by index 'obj[y]'). Instead of giving
the data in *x* and *y*, you can provide the object in the *data*
parameter and just give the labels for *x* and *y*:

>>> plot('xlabel', 'ylabel', data=obj)

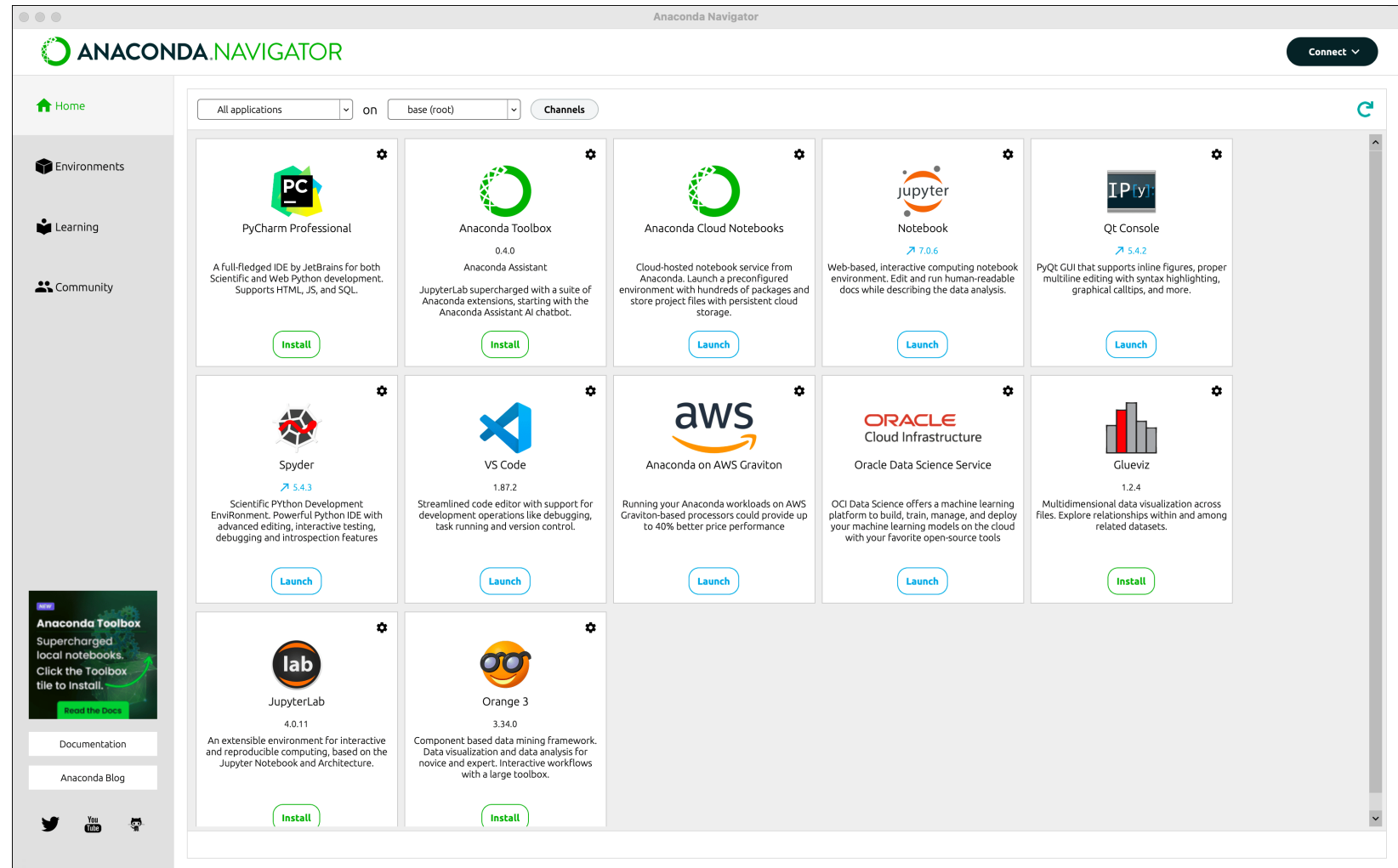
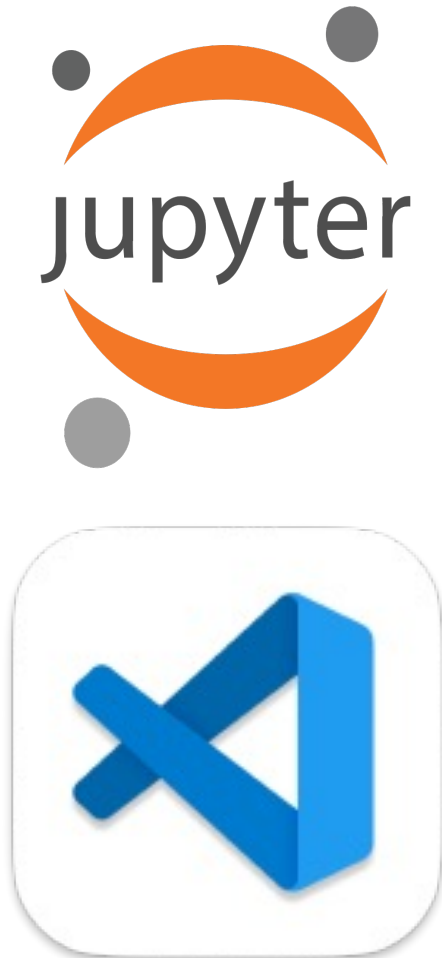
All indexable objects are supported. This could e.g. be a 'dict', a
pandas.DataFrame or a structured numpy array.

**Plotting multiple sets of data**

There are various ways to plot multiple sets of data.

- The most straight forward way is just to call 'plot' multiple times.
```

Environment(s)



Hands on session

p1_tutorial.ipynb > M Short NumPy and scikit-image Tutorial

Jupyter Notebook

+ Code + Markdown | Run All Clear All Outputs | Outline ...

Select Kernel

▶ ✎ 📄 ... 🗑

Short NumPy and scikit-image Tutorial

Course: Computer Vision

This is a very short introduction to [numpy](#) and [Scikit-image library](#), including the main operations for image manipulation.

Numpy is the most important scientific package in the Python ecosystem, because it provides a common datastructure on which many other packages are build on.

What is an image? Images are n-dimensional arrays (`ndarray`) where each element corresponds to a pixel.

The most common image kinds are:

1. **Binary images**, containing 1 channel of X rows and Y columns. They use logical (True/False, 0/1...) values.
2. **Grey-level images**, containing 1 channel of X rows and Y columns. They use a set of values, depending on the data type (float, uint8, uint16...)
3. **RGB /color images**, containing 3 channels per color (RGB). Sometimes, **RGBA (4 channels)** images are used.

Images manipulated by the Scikit-image library are stored as **Numpy arrays**.

See [A crash course on NumPy for images](#) for more information.

Other libraries we will use:

- **Matplotlib** Matplotlib is a comprehensive library for creating static, animated, and interactive visualizations in Python.
- **Scikit-learn** Simple and efficient tools for predictive data analysis.
- **Scipy** SciPy provides algorithms for optimization, integration, interpolation, eigenvalue problems, algebraic equations, differential equations, statistics and many other classes of problems.
- **OpenCV** OpenCV library is an open source computer vision and machine learning software library.
- **Pillow** Pillow (PIL) is an acronym for Python Imaging Library.